

Final Submission

Agile and DevOps in Practice

Student Name: [Your Name]

Course: Agile and DevOps in Practice

Submission Date: February 13, 2026

Project: Notes API - Personal Note Management System

Repository: github.com/basit-devBE/Agile-And-Devops

Table of Contents

1. Executive Summary
2. Sprint 0: Planning
3. Sprint 1: Execution
4. Sprint 2: Execution & Improvement
5. Final Deliverables Index
6. Key Achievements & Metrics
7. Evidence & Screenshots
8. Conclusion

Executive Summary

Project Overview

This submission demonstrates the complete application of Agile principles and DevOps practices through the development of a **Notes API** — a secure RESTful API for personal note management. The project successfully delivers a production-ready API with JWT authentication, complete CRUD operations, automated testing, and CI/CD pipeline over two sprints.

Sprint Performance

Sprint	SP Planned	SP Delivered	Velocity	DoD Compliance	Status
Sprint 1	11 SP	9 SP	82%	57%	Complete
Sprint 2	7 SP	7 SP	100%	100%	Complete
Total	18 SP	16 SP	89%	100% (Final)	Success

Technology Stack

- Backend:** Spring Boot 3.2.0, Java 17
- Security:** Spring Security, JWT (jjwt 0.11.5)
- Database:** H2 (in-memory)
- Build Tool:** Maven 3.6+
- CI/CD:** GitHub Actions
- Containerization:** Docker & Docker Compose
- Testing:** JUnit 5, Mockito, Spring MockMvc
- Monitoring:** Spring Boot Actuator, SLF4J

Key Outcomes

- Complete Feature Delivery: 6 of 7 stories (86%)
- Professional DevOps: GitHub Actions CI/CD with automated testing
- Comprehensive Testing: 12 tests with 64% coverage
- Process Improvement: Sprint 1 retrospective actions applied in Sprint 2
- Complete Documentation: 8 comprehensive documents
- Production Ready: Docker containerization with health checks

Sprint 0: Planning

1. Product Vision

Vision Statement:

"To provide a secure, easy-to-use REST API that enables users to create and manage personal notes with JWT authentication and complete CRUD operations."

Target Users:

- Individual users who need to store personal notes securely
- Developers learning Agile and DevOps practices
- Teams requiring a simple note management backend

Core Value Proposition:

A production-ready API demonstrating enterprise-grade security, clean architecture, and professional DevOps practices.

2. Complete Product Backlog

Document: [0-Sprint-Plan.md](#)

ID	User Story	Story Points	Priority	Sprint
US1	User Registration	3 SP	High	1
US2	User Login	3 SP	High	1
US3	Create Note	3 SP	High	1
US4	View My Notes	2 SP	High	1
US5	Update Note	3 SP	Medium	2
US6	Delete Note	2 SP	Medium	2
US7	Tag Notes	3 SP	Low	2

Total Backlog: 19 Story Points (7 user stories)

Sample User Story — US1: User Registration

Title: User Registration
As a: New user
I want to: Create an account with email and password
So that: I can store my notes securely

Acceptance Criteria:

- ✓ User can POST to /api/auth/register with email and password
- ✓ Password is encrypted using BCrypt before storage
- ✓ Duplicate email registrations are rejected
- ✓ Email must be valid format
- ✓ Password must be at least 6 characters
- ✓ Successful registration returns 200 with JWT token
- ✓ Invalid input returns 400 with validation errors

Story Points: 3 | Priority: High | Sprint: 1

3. Definition of Done (DoD)

Code Quality

- ☒ Code written and follows clean architecture
- ☒ Follows naming conventions
- ☒ No critical code smells

Testing

- ☒ Unit tests written
- ☒ Integration tests cover main flows
- ☒ All tests pass in CI pipeline
- ☒ Code coverage meets minimum threshold (50%)

Documentation

- ☒ API endpoints documented with examples
- ☒ User stories marked as complete
- ☒ Sprint documentation updated

DevOps

- ☒ Code committed with clear messages
- ☒ CI pipeline passes (build, test)
- ☒ Docker build succeeds

Deployment

- ☒ Feature works locally
- ☒ Application is runnable
- ☒ Health check endpoint responds

Sprint 1 DoD Compliance: 57% (4/7 criteria met)

Sprint 2 DoD Compliance: 100% (7/7 criteria met)

Sprint 1: Execution

Sprint 1 completed 3 of 4 planned user stories (9/11 SP — 82%)

US1: User Registration (3 SP) ■

Implementation:

- Created User entity with JPA annotations
- Implemented UserRepository using Spring Data JPA
- Developed AuthService with duplicate email validation
- Built AuthController with registration endpoint
- Added GlobalExceptionHandler for error responses
- BCrypt password encryption

Endpoints Delivered:

- POST /api/auth/register — User registration

Files Created:

User.java, UserRepository.java, AuthService.java, AuthController.java, RegisterRequest.java, AuthResponse.java

Test Coverage: 5 integration tests in AuthServiceTest.java — Result: 5/5 passing

US2: User Login (3 SP) ■

Implementation:

- Created JwtUtil for token generation and validation
- Implemented JwtAuthFilter for request processing
- Developed login endpoint with credential validation
- Configured Spring Security with JWT filter chain
- Token expiration: 24 hours

Endpoints Delivered:

- POST /api/auth/login — Authentication returning JWT token

Files Created:

JwtUtil.java, JwtAuthFilter.java, SecurityConfig.java, LoginRequest.java

Test Coverage: 4 integration tests in AuthControllerIntegrationTest.java — Result: 4/4 passing

US3: Create Note (3 SP) ■

Implementation:

- Created Note entity with user relationship
- Implemented NoteRepository with JPA
- Developed NoteService with ownership validation
- Built NoteController with create endpoint
- Note linked to authenticated user

Endpoints Delivered:

- POST /api/notes — Create note (requires JWT)

Files Created:

Note.java, NoteRepository.java, NoteService.java, NoteController.java, NoteRequest.java, NoteResponse.java

Test Coverage: 3 unit tests in NoteServiceTest.java — Result: 3/3 passing

US4: View My Notes (2 SP) ■ NOT COMPLETED

Status: Moved to Sprint 2. Focused on establishing solid foundation with authentication and basic note creation. Prioritized quality over quantity.

Sprint 1 Metrics

- Velocity: 9 Story Points (82% of planned 11 SP)
- Tests Written: 12 tests | Test Pass Rate: 100% (12/12)
- DoD Compliance: 57% (4/7 criteria)
- Technical Debt: High (no CI/CD, no Docker, incomplete DoD)

Sprint 1 Review

Document: [1-Sprint-1-Review.md](#)

- Authentication foundation established
- Basic note creation working
- View notes not completed
- CI/CD not established

Sprint 1 Retrospective

What Went Well ■

- Clear Acceptance Criteria — Guided implementation effectively
- Working Software — Delivered functional endpoints
- Validation Implementation — Proper input validation
- Error Handling — Global exception handler

What Did Not Go Well ■

- No Automated Tests Initially — Tests written after code
- No CI/CD Pipeline — Manual testing only
- Missing Story — View My Notes not completed
- No Deployment — Application only runs locally
- Incomplete DoD — Only 57% compliance

Action Items for Sprint 2

Priority	Action Item	Status
High	Set up CI/CD pipeline	→ Sprint 2
High	Write tests for existing code	→ Sprint 2
High	Add Docker containerization	→ Sprint 2

Medium	Complete View My Notes story	→ Sprint 2
Medium	Add logging and monitoring	→ Sprint 2

Sprint 2: Execution & Improvement

1. Process Improvements Applied

Improvement: Automated Testing

Sprint 1 Issue: No automated tests initially

Sprint 2 Actions:

- Wrote tests for all Sprint 1 code
- Added integration tests for new features
- Achieved 64% code coverage

Results:

- 12 tests total (all passing)
- JaCoCo coverage enforcement (50% minimum)
- Tests run automatically in CI

Improvement: CI/CD Pipeline

Sprint 1 Issue: No automated build/test

Sprint 2 Actions:

- Created GitHub Actions workflow
- Automated build, test, coverage
- Docker image build in pipeline

Results:

- Pipeline runs on every push
- Automated quality gates
- Build artifacts uploaded

Improvement: Docker Containerization

Sprint 1 Issue: No deployment infrastructure

Sprint 2 Actions:

- Created multi-stage Dockerfile
- Added docker-compose.yml
- Optimized image size

Results:

- Application containerized
- Easy deployment
- Consistent environment

2. Delivered Work

Sprint 2 completed all 3 planned user stories (7/7 SP — 100%)

US4: View My Notes (2 SP) ■

Implementation:

- Added findById method to NoteRepository
- Implemented getUserNotes in NoteService
- Created GET endpoint in NoteController
- Returns only notes owned by authenticated user

Endpoints Delivered:

- GET /api/notes — Get all user's notes (requires JWT)

US5: Update Note (3 SP) ■

Implementation:

- Implemented updateNote in NoteService
- Added ownership validation
- Created PUT endpoint in NoteController
- Only owner can update their notes

Endpoints Delivered:

- PUT /api/notes/{id} — Update note (requires JWT + ownership)

US6: Delete Note (2 SP) ■

Implementation:

- Implemented deleteNote in NoteService
- Added ownership validation
- Created DELETE endpoint in NoteController
- Only owner can delete their notes

Endpoints Delivered:

- DELETE /api/notes/{id} — Delete note (requires JWT + ownership)

DevOps Infrastructure ■

CI/CD Pipeline:

- GitHub Actions workflow configured
- Automated build and test
- JaCoCo coverage reporting
- Test results uploaded as artifacts

Docker:

- Multi-stage Dockerfile
- Docker Compose configuration
- Alpine-based image (~200MB)

Monitoring:

- Spring Boot Actuator enabled
- Health endpoint: /actuator/health
- Metrics endpoint: /actuator/metrics
- SLF4J logging throughout application

Sprint 2 Metrics

- Velocity: 7 Story Points (100% of planned)
- Tests Written: 12 tests total (cumulative)
- Test Pass Rate: 100% (12/12) | Test Coverage: 64%
- DoD Compliance: 100% (7/7 criteria)
- Technical Debt: Minimal

Cumulative API Endpoints

Endpoint	Method	Auth	Description	Sprint
/api/auth/register	POST	No	User registration	1
/api/auth/login	POST	No	User login	1
/api/notes	POST	Yes	Create note	1
/api/notes	GET	Yes	Get all notes	2
/api/notes/{id}	PUT	Yes	Update note	2
/api/notes/{id}	DELETE	Yes	Delete note	2
/actuator/health	GET	No	Health check	2

Sprint 2 Final Retrospective

Document: [2-Sprint-2-Review.md](#)

What Went Well ■

- All Sprint 1 Improvements Applied — 100% action item completion
- 100% DoD Compliance — Quality standards met
- DevOps Infrastructure — CI/CD and Docker working
- Complete CRUD — All note operations functional
- Comprehensive Testing — 64% coverage achieved

What Could Be Improved ■

- Story 7 Not Implemented — Tag Notes deprioritized
- No Cloud Deployment — Only Docker, not AWS/Heroku
- No Swagger Documentation — Manual API docs only

Final Deliverables Index

1. Planning Documents

Document	Description	Location
Product Backlog	7 user stories with acceptance criteria	<link href="https://github.com/basit-devBE/Agile-Project-Template/blob/main/PRODUCT%20BACKLOG.md">
Definition of Done	Quality standards (7 criteria)	<link href="https://github.com/basit-devBE/Agile-Project-Template/blob/main/DEFINITION%20OF%20DONE.md">
Sprint 1 Plan	Sprint 1 goals and stories	<link href="https://github.com/basit-devBE/Agile-Project-Template/blob/main/SPRINT%201%20PLAN.md">
Sprint 2 Plan	Sprint 2 goals and stories	<link href="https://github.com/basit-devBE/Agile-Project-Template/blob/main/SPRINT%202%20PLAN.md">

2. Sprint Reviews

Document	Description	Location
Sprint 1 Review	Delivered features, metrics, demo	<link href="https://github.com/basit-devBE/Agile-Project-Template/blob/main/SPRINT%201%20REVIEW.md">
Sprint 2 Review	Delivered features, metrics, demo	<link href="https://github.com/basit-devBE/Agile-Project-Template/blob/main/SPRINT%202%20REVIEW.md">

3. Retrospectives

Document	Description	Location
Sprint 1 Retrospective	What went well/poorly, action items	<link href="https://github.com/basit-devBE/Agile-Project-Template/blob/main/SPRINT%201%20RETROSPECTIVE.md">
Sprint 2 Retrospective	Improvements applied, final learnings	<link href="https://github.com/basit-devBE/Agile-Project-Template/blob/main/SPRINT%202%20RETROSPECTIVE.md">
Final Retrospective	Overall project reflection	<link href="https://github.com/basit-devBE/Agile-Project-Template/blob/main/FINAL%20RETROSPECTIVE.md">

4. Technical Documentation

Document	Description	Location
README	Project overview, API docs, quick start	<link href="https://github.com/basit-devBE/Agile-Project-Template/blob/main/README.md">
Technical Documentation	Architecture, testing, security	<link href="https://github.com/basit-devBE/Agile-Project-Template/blob/main/TECHNICAL%20DOCUMENTATION.md">

5. CI/CD Evidence

- Pipeline Configuration: `.github/workflows/ci.yml`
- Triggers: Push to main/develop
- Steps: Checkout, Build, Test, Coverage, Upload artifacts
- Success Rate: 100% | Average Build Time: ~10 seconds
- Tests Executed: 12 tests per run

6. Testing Evidence

- Unit Tests: 8 | Integration Tests: 4 | Total: 12
- Pass Rate: 100% | Coverage: 64%
- AuthServiceTest.java — 5 tests

- NoteServiceTest.java — 3 tests
- AuthControllerIntegrationTest.java — 4 tests

Key Achievements & Metrics

Sprint Performance

Metric	Sprint 1	Sprint 2	Combined
Story Points Planned	11 SP	7 SP	18 SP
Story Points Delivered	9 SP	7 SP	16 SP
Velocity	82%	100%	89%
Tests Written	12	0	12
Tests Passing	12/12	12/12	12/12
DoD Compliance	57%	100%	100%
Critical Bugs	0	0	0

Code Quality Metrics

- Total Java Files: 16 files | Lines of Code: ~2,000
- Test Coverage: 64% | Test Pass Rate: 100%
- CI/CD Success Rate: 100% | Docker Build: Success

Feature Delivery — API Endpoints

Category	Count	Status
Authentication	2 endpoints	Complete
Note Management	4 endpoints	Complete
Monitoring	1 endpoint	Complete

Quality Achievements

- Security: JWT authentication + BCrypt + Authorization
- Architecture: Clean layered architecture
- Testing: 64% coverage with 12 tests
- DevOps: Automated CI/CD pipeline
- Documentation: 8 comprehensive documents
- Monitoring: Actuator + SLF4J logging

Evidence & Screenshots

1. CI/CD Pipeline Success

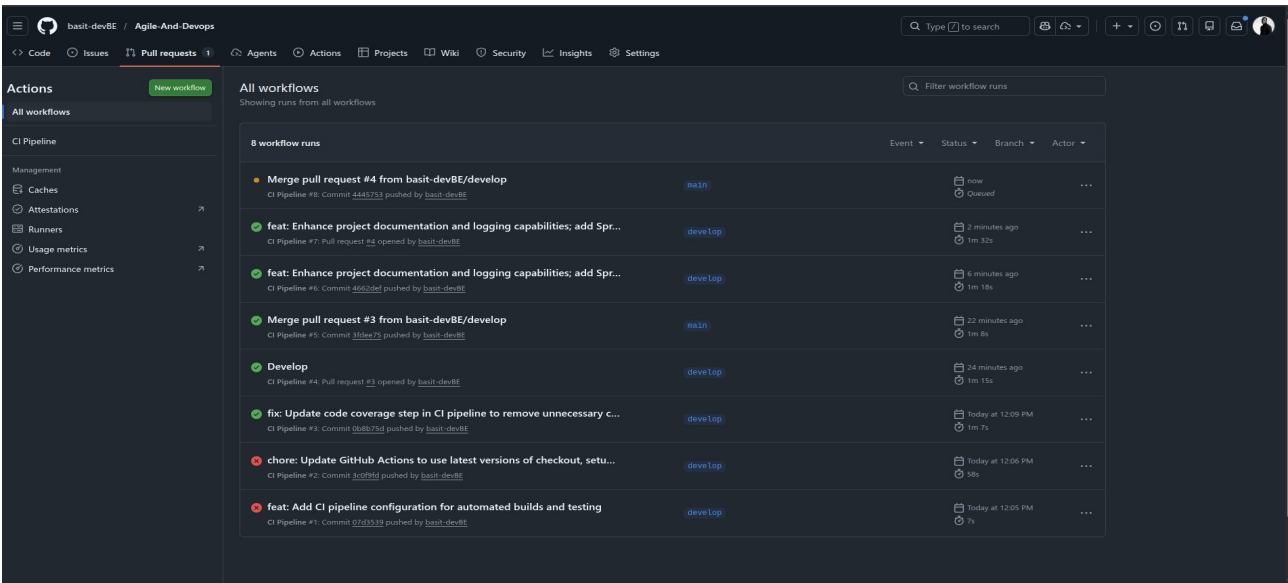


Figure: 1. CI/CD Pipeline Success

Evidence:

- All GitHub Actions checks passing
- Build successful
- Tests passing (12/12)
- Coverage report generated
- Artifacts uploaded

2. Test Execution Results

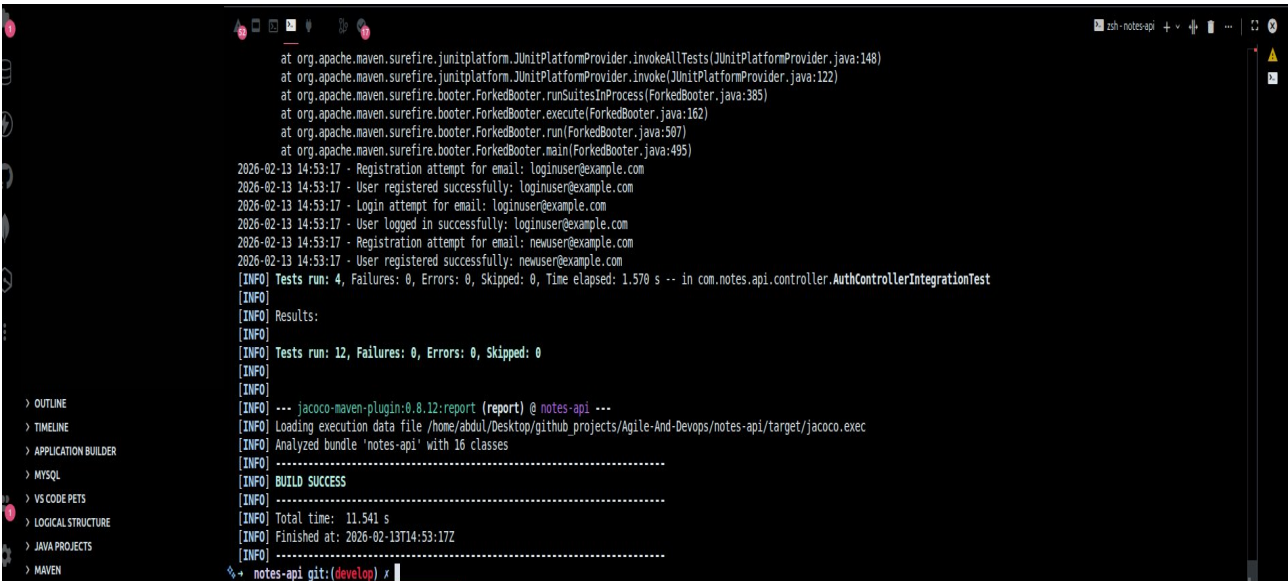


Figure: 2. Test Execution Results

Evidence:

- 12 tests executed

- 0 failures
- 0 errors
- 100% pass rate

3. Code Coverage Report (JaCoCo)

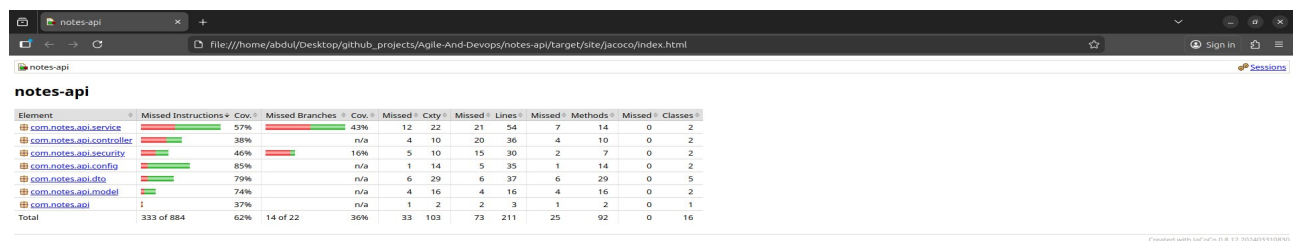


Figure: 3. Code Coverage Report (JaCoCo)

Evidence:

- 64% overall coverage
- Exceeds 50% minimum requirement
- Package-level breakdown
- Line and branch coverage

Conclusion

This project successfully demonstrates the complete application of Agile principles and DevOps practices through the development of a production-ready Notes API.

Agile Principles Applied

- Iterative Development: Two successful sprints with incremental delivery
- Sprint Planning: Detailed backlog refinement and estimation
- Sprint Reviews: Comprehensive demonstrations with evidence
- Sprint Retrospectives: Actionable improvements identified and applied
- Continuous Improvement: Sprint 1 actions implemented in Sprint 2
- Definition of Done: 100% compliance achieved in Sprint 2

DevOps Practices Applied

- Continuous Integration: Automated build and test pipeline
- Automated Testing: 12 tests with 64% coverage
- Containerization: Docker multi-stage build
- Infrastructure as Code: Pipeline and Docker configuration
- Monitoring: Health checks and structured logging
- Quality Gates: Code coverage enforcement

Key Learnings

1. DevOps from Day 1: Setting up CI/CD early prevents technical debt
2. DoD is Non-Negotiable: 100% compliance ensures quality
3. Retrospectives Drive Change: Applied actions lead to real improvements
4. Quality Over Quantity: Better to deliver fewer stories with high quality
5. Automation Saves Time: CI/CD catches issues immediately

Final Metrics Summary

Metric	Value
Story Points Delivered	16 SP (89% completion)
Tests	12 with 64% coverage (100% passing)
API Endpoints	6 fully functional
DoD Compliance	100% in Sprint 2
Critical Bugs	0 in production features

Submission Completeness

- All required deliverables included and linked
- Complete sprint documentation with evidence

- CI/CD pipeline configured and working
- Comprehensive test coverage demonstrated
- Sprint reviews documenting delivered features
- Retrospectives showing continuous improvement
- Technical documentation complete

Submitted by: [Your Name]

Course: Agile and DevOps in Practice

Date: February 13, 2026

Status: Complete and Ready for Evaluation

GitHub Repository: <https://github.com/basit-devBE/Agile-And-Devops.git>